

SQL Subquery Practice — Complete Solutions

Objective

Use subqueries for data analysis in MySQL.

Setup

Create Tables

```
```sql
CREATE TABLE employees (
 employee_id INT PRIMARY KEY,
 name VARCHAR(50),
 department VARCHAR(50)
);

CREATE TABLE salaries (
 employee_id INT,
 salary DECIMAL(10, 2),
 FOREIGN KEY (employee_id)
 REFERENCES employees(employee_id)
);
```

### Insert Sample Data

```
INSERT INTO employees VALUES
(1, 'Alice', 'HR'),
(2, 'Bob', 'Engineering'),
(3, 'Charlie', 'Finance');
```

```
INSERT INTO salaries VALUES
(1, 50000.00),
(2, 70000.00),
(3, 60000.00);
```

---

## Basic Question

### Employees with Salary Higher Than Average

```

SELECT name,
 department
FROM employees
WHERE employee_id IN (
 SELECT employee_id
 FROM salaries
 WHERE salary > (
 SELECT AVG(salary)
 FROM salaries
)
);

```

### Expected Result

name	department
Bob	Engineering

## 1. Employees with the Highest Salary in Their Department

Use a correlated subquery.

```

SELECT e.name,
 e.department,
 s.salary
FROM employees e
JOIN salaries s
 ON e.employee_id = s.employee_id
WHERE s.salary = (
 SELECT MAX(s2.salary)
 FROM employees e2
 JOIN salaries s2
 ON e2.employee_id = s2.employee_id
 WHERE e2.department = e.department
);

```

### Explanation

The inner query finds the maximum salary for the current employee's department.

WHERE e2.department = e.department

This makes the query a correlated subquery because the inner query refers to the outer query.

---

## 2. Total Salary Paid to Employees in Each Department

```
SELECT e.department,
 (
 SELECT SUM(s.salary)
 FROM employees e2
 JOIN salaries s
 ON e2.employee_id = s.employee_id
 WHERE e2.department = e.department
) AS total_salary
FROM employees e
GROUP BY e.department;
```

### Expected Result

department	total_salary
Engineering	70000.00
Finance	60000.00
HR	50000.00

---

## 3. Employees Who Earn Less Than Their Departmental Average

```
SELECT e.name,
 e.department,
 s.salary
FROM employees e
JOIN salaries s
 ON e.employee_id = s.employee_id
WHERE s.salary < (
```

```

SELECT AVG(s2.salary)
FROM employees e2
JOIN salaries s2
 ON e2.employee_id = s2.employee_id
WHERE e2.department = e.department
);

```

## Explanation

The subquery calculates the average salary for the employee's department.

The outer query returns employees whose salary is lower than that departmental average.

With the current sample data, each department has only one employee, so this query returns no rows.

# 4. Combine Subqueries with JOIN

Find employees earning more than the overall average salary.

```

SELECT e.employee_id,
 e.name,
 e.department,
 s.salary
FROM employees e
JOIN salaries s
 ON e.employee_id = s.employee_id
JOIN (
 SELECT AVG(salary) AS average_salary
 FROM salaries
) avg_data
 ON s.salary > avg_data.average_salary;

```

## Expected Result

employee_id	name	department	salary
2	Bob	Engineering	70000.00

## 5. Employees Whose Salary Is in the Top 10%

A ranking-based approach:

```
SELECT e.name,
 e.department,
 s.salary
FROM employees e
JOIN salaries s
 ON e.employee_id = s.employee_id
WHERE (
 SELECT COUNT(*)
 FROM salaries s2
 WHERE s2.salary > s.salary
) < CEIL(
 (SELECT COUNT(*) FROM salaries) * 0.10
);
```

### Explanation

The correlated subquery counts how many employees have a salary greater than the current employee's salary.

The result is compared with the number of records representing the top 10%.

With only three employees, a top-10% calculation is not very meaningful. This query is more useful with a larger dataset.

---

## 6. Calculate Each Employee's Salary Rank

Use a subquery in the **SELECT** clause.

```
SELECT e.name,
 e.department,
 s.salary,
 (
 SELECT COUNT(*) + 1
 FROM salaries s2
 WHERE s2.salary > s.salary
) AS rank
```

```

 WHERE s2.salary > s.salary
) AS salary_rank
FROM employees e
JOIN salaries s
 ON e.employee_id = s.employee_id
ORDER BY salary_rank;

```

### Expected Result

name	department	salary	salary_rank
Bob	Engineering	70000.0 0	1
Charlie	Finance	60000.0 0	2
Alice	HR	50000.0 0	3

### Explanation

For each employee, the subquery counts employees earning more.

For Bob:

Employees earning more = 0  
 Rank = 0 + 1 = 1

For Charlie:

Employees earning more = 1  
 Rank = 1 + 1 = 2

For Alice:

Employees earning more = 2  
 Rank = 2 + 1 = 3

---

## 7. Identify Duplicate Entries in the Salaries Table

## Find Employees with Multiple Salary Records

```
SELECT employee_id,
 COUNT(*) AS duplicate_count
FROM salaries
GROUP BY employee_id
HAVING COUNT(*) > 1;
```

## Find the Actual Duplicate Records

```
SELECT s.*
FROM salaries s
WHERE s.employee_id IN (
 SELECT employee_id
 FROM salaries
 GROUP BY employee_id
 HAVING COUNT(*) > 1
);
```

## Find Exact Duplicate Rows

If duplicate means the same `employee_id` and the same `salary`:

```
SELECT employee_id,
 salary,
 COUNT(*) AS duplicate_count
FROM salaries
GROUP BY employee_id, salary
HAVING COUNT(*) > 1;
```

---

## Complete SQL Script

```
-- =====
-- SQL SUBQUERY PRACTICE
-- =====

-- Create employees table
CREATE TABLE employees (
 employee_id INT PRIMARY KEY,
 name VARCHAR(50),
 department VARCHAR(50)
);
```

```
-- Create salaries table
CREATE TABLE salaries (
 employee_id INT,
 salary DECIMAL(10, 2),
 FOREIGN KEY (employee_id)
 REFERENCES employees(employee_id)
);
```

```
-- Insert employees
INSERT INTO employees VALUES
(1, 'Alice', 'HR'),
(2, 'Bob', 'Engineering'),
(3, 'Charlie', 'Finance');
```

```
-- Insert salaries
INSERT INTO salaries VALUES
(1, 50000.00),
(2, 70000.00),
(3, 60000.00);
```

```
-- =====
-- Basic Question
-- Employees earning more than average salary
-- =====
```

```
SELECT name,
 department
FROM employees
WHERE employee_id IN (
 SELECT employee_id
 FROM salaries
 WHERE salary > (
 SELECT AVG(salary)
 FROM salaries
)
);
```

```
-- =====
-- 1. Highest salary in each department
-- Correlated Subquery
-- =====
```



```

SELECT e.name,
 e.department,
 s.salary
FROM employees e
JOIN salaries s
 ON e.employee_id = s.employee_id
WHERE s.salary = (
 SELECT MAX(s2.salary)
 FROM employees e2
 JOIN salaries s2
 ON e2.employee_id = s2.employee_id
 WHERE e2.department = e.department
);

```

```

-- =====
-- 2. Total salary by department
-- =====

```

```

SELECT e.department,
 (
 SELECT SUM(s.salary)
 FROM employees e2
 JOIN salaries s
 ON e2.employee_id = s.employee_id
 WHERE e2.department = e.department
) AS total_salary
FROM employees e
GROUP BY e.department;

```

```

-- =====
-- 3. Employees earning less than
-- departmental average
-- =====

```

```

SELECT e.name,

```

---

## Key Concepts Covered

Question	SQL Concept
Basic	Nested subquery
1	Correlated subquery + <b>MAX()</b>
2	Correlated subquery + <b>SUM()</b>
3	Correlated subquery + <b>AVG()</b>
4	Derived table + <b>JOIN</b>
5	Subquery + ranking
6	Scalar subquery in <b>SELECT</b>
7	Subquery + <b>GROUP BY</b> + <b>HAVING</b>

---

## Subquery Types

### Subquery

